UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

Clase Práctica 6: Recuperación

Normalización - Ejercicio Pendiente

Ejercicio pendiente - normalización

Dada una relación
{CUIL, DNI, numeroPuntoDeVenta, montoFacturado} que indica los

detalles de los puntos de venta para la facturación de los monotributistas.

¿Está en 3FN? ¿Y en BCNF?

Ejercicio pendiente - normalización

R = {CUIL, DNI, numeroPuntoDeVenta, montoFacturado}

Claves = {(CUIL, numeroPuntoDeVenta), (DNI, numeroPuntoDeVenta)}

Ejercicio pendiente - normalización

```
R = {CUIL, DNI, numeroPuntoDeVenta, montoFacturado}

Claves = {(CUIL, numeroPuntoDeVenta), (DNI, numeroPuntoDeVenta)}

Dependencias Funcionales:
    CUIL -> DNI
    DNI  -> CUIL
    {CUIL, numeroPuntoDeVenta} -> monto
    {DNI,  numeroPuntoDeVenta} -> monto
```

Ejercicio pendiente - normalización

```
R = {CUIL, DNI, numeroPuntoDeVenta, montoFacturado}

Claves = {(CUIL, numeroPuntoDeVenta), (DNI, numeroPuntoDeVenta)}

Dependencias Funcionales:
    CUIL -> DNI
    DNI  -> CUIL
    {CUIL, numeroPuntoDeVenta} -> monto
    {DNI,  numeroPuntoDeVenta} -> monto

BCNF?
    “Toda DF depende de una superclave”
    Superclave: X -> R entonces X es superclave
```

Ejercicio pendiente - normalización

```
R = {CUIL, DNI, numeroPuntoDeVenta, montoFacturado}

Claves = {(CUIL, numeroPuntoDeVenta), (DNI, numeroPuntoDeVenta)}

Dependencias Funcionales:
    CUIL -> DNI
    DNI  -> CUIL
    {CUIL, numeroPuntoDeVenta} -> monto
    {DNI,  numeroPuntoDeVenta} -> monto

BCNF?
    “Toda DF depende de una superclave”
    Superclave: X -> R entonces X es superclave

    CUIL -> DNI no depende de una superclave
    => no esta en BCNF
```

Ejercicio pendiente - normalización

3FN?

“No hay dependencias transitivas de la CC”

Una DF es transitiva cuando existe Z tal que X -> Z, Z -> Y (no trivial) y no se cumple que Z -> X

Ejercicio pendiente - normalización

3FN?

“No hay dependencias transitivas de la CC”

Una DF es transitiva cuando existe Z tal que X -> Z, Z -> Y (no trivial) y no se cumple que Z -> X

Otra forma:

“Cumple 2FN y ningún atributo no primo tiene dependencia transitiva de una CC”

X es primo si es miembro de una CC
Primos = {CUIL, DNI, numeroPuntoVenta}
No primos = {monto}

Ejercicio pendiente - normalización

```
3FN?
    “No hay dependencias transitivas de la CC”

    Una DF es transitiva cuando existe Z tal que X -> Z, Z -> Y
    (no trivial) y no se cumple que Z -> X

Otra forma:
    “Cumple 2FN y ningún atributo no primo tiene dependencia
    transitiva de una CC”

    X es primo si es miembro de una CC
    Primos = {CUIL, DNI, numeroPuntoVenta}
    No primos = {monto}

    => monto no tiene dependencia transitiva con una CC
    => esta en 3FN
```

Ejercicio pendiente - normalización

2FN?

“No hay dependencias parciales de la CC”

Decimos que X->Y es **DF parcial** si para algún B, {X sin B} -> Y

Ejercicio pendiente - normalización

2FN?

“No hay dependencias parciales de la CC”

Decimos que X->Y es **DF parcial** si para algún B, {X sin B} -> Y

{CUIL, numeroPuntoDeVenta} -> monto
{DNI, numeroPuntoDeVenta} -> monto

Ejercicio pendiente - normalización

2FN?

“No hay dependencias parciales de la CC”

Decimos que X->Y es **DF parcial** si para algún B, {X sin B} -> Y

{CUIL, numeroPuntoDeVenta} -> monto
{DNI, numeroPuntoDeVenta} -> monto

Si saco alguna parte izquierda ya no cumple la DF
=> no hay dependencias parciales
=> está en 2FN

Transacciones - Repaso

Repaso transacciones: definición básica

$$T_i \subseteq \{w_i(X);\ r_i(X)\} \cup \{c_i,\ a_i\}$$

- T_i representa la transacción i y $<_i$ es el orden parcial de T_i
- $w_i(X)$ escritura de la transacción i sobre el ítem X
- $r_i(X)$ lectura por la transacción i del ítem X
- c_i representa el *commit* de la transacción i
- a_i representa el *abort* de la transacción i
- $a_i \in T_i \Leftrightarrow c_i \notin T_i$
- para cualquier operación $o_i \in T_i$ entonces $o_i <_i a_i$ ó $o_i <_i c_i$

$$T_1 = w_1(B);\ r_1(C);\ w_1(C);\ c_1$$
$$T_2 = r_2(D);\ r_2(B);\ w_2(C);\ c_2$$

Repaso transacciones: equivalencia de historias

Operaciones Conflictivas: Dos operaciones son conflictivas si operan sobre el mismo ítem y al menos una de ellas es una escritura.

Repaso transacciones: equivalencia de historias

Operaciones Conflictivas: Dos operaciones son conflictivas si operan sobre el mismo ítem y al menos una de ellas es una escritura.

Equivalencia: Dos historias H_i y H_j son conflicto equivalentes $H_i \equiv H_j$

1. Están definidas sobre el mismo conjunto de transacciones
2. El orden de las operaciones conflictivas de transacciones no abortadas es el mismo.

Repaso transacciones: equivalencia de historias

Operaciones Conflictivas: Dos operaciones son conflictivas si operan sobre el mismo ítem y al menos una de ellas es una escritura.

Equivalencia: Dos historias H_i y H_j son conflicto equivalentes $H_i \equiv H_j$

1. Están definidas sobre el mismo conjunto de transacciones
2. El orden de las operaciones conflictivas de transacciones no abortadas es el mismo.

Una historia H es serializable (SR) si es conflicto equivalente a alguna historia serial H_s, es decir $H \equiv H_s$

Repaso serializabilidad: testeo

Grafo de precedencia

Se utiliza el grafo de precedencia SG(H). Es un grafo dirigido con las siguientes características:

Un nodo para cada transacción $T_i \subseteq$ H Hay ejes entre T_i y T_j sí y sólo sí hay una operación de T_i que precede en H a una operación de T_j y son operaciones conflictivas. Etiquetamos los ejes del grafo con los nombres de los ítems que los generan.

Teorema de la seriabilidad

Una historia H es **SR** si y solo si **SG(H) es acíclico.**

Repaso serializabilidad: ejemplo

Equivalencia serial

Si el SG(H) es acíclico entonces los órdenes seriales equivalentes son los diferentes órdenes topológicos del grafo.

Repaso serializabilidad: ejemplo

Equivalencia serial

Si el SG(H) es acíclico entonces los órdenes seriales equivalentes son los diferentes órdenes topológicos del grafo.

¿Son las siguientes Hs serializables?

H_1 = R_1(A); R_1(B); R_2(B); R_2(B)

Repaso serializabilidad: ejemplo

Equivalencia serial

Si el SG(H) es acíclico entonces los órdenes seriales equivalentes son los diferentes órdenes topológicos del grafo.

¿Son las siguientes Hs serializables?

H_1 = R_1(A); R_1(B); R_2(B); R_2(B)

H_2 = R_1(B); W_1(B)

Repaso serializabilidad: ejemplo

Equivalencia serial

Si el SG(H) es acíclico entonces los órdenes seriales equivalentes son los diferentes órdenes topológicos del grafo.

¿Son las siguientes Hs serializables?

H_1 = R_1(A); R_1(B); R_2(B); R_2(B)

H_2 = R_1(B); W_1(B)

H_3 = W_1(A); R_2(A); W_3(A)

Repaso serializabilidad: ejemplo

Equivalencia serial

Si el SG(H) es acíclico entonces los órdenes seriales equivalentes son los diferentes órdenes topológicos del grafo.

¿Son las siguientes Hs serializables?

H_1 = R_1(A); R_1(B); R_2(B); R_2(B)

H_2 = R_1(B); W_1(B)

H_3 = W_1(A); R_2(A); W_3(A)

H_4 = R_1(X); R_2(X); W_1(Y); R_3(Y); R_2(Y); W_2(X); R_3(W); W_3(Y); R_4(W); R_4(Z); R_1(Z); W_1(Z)

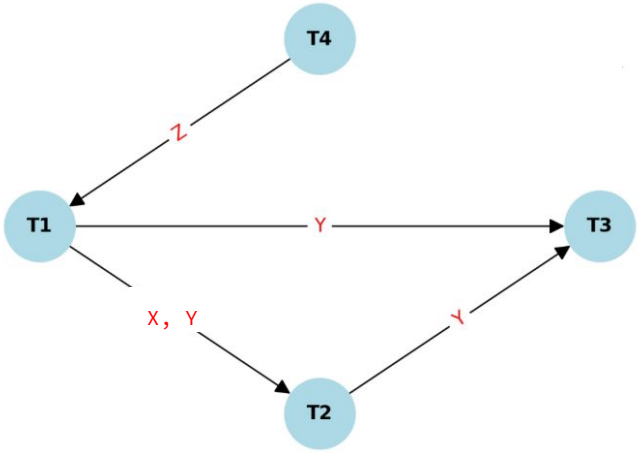
Repaso serializabilidad: ejemplo

H_4 = R_1(X); R_2(X); W_1(Y); R_3(Y); R_2(Y); W_2(X); R_3(W); W_3(Y); R_4(W); R_4(Z); R_1(Z); W_1(Z)

T_1	T_2	T_3	T_4
$r_1(X)$			
	$r_2(X)$		
$w_1(Y)$			
		$r_3(Y)$	
	$r_2(Y)$		
	$w_2(X)$		
		$r_3(W)$	
		$w_3(Y)$	
			$r_4(W)$
			$r_4(Z)$
$r_1(Z)$			
$w_1(Z)$			

Repaso serializabilidad: ejemplo

H_4 = R_1(X); R_2(X); W_1(Y); R_3(Y); R_2(Y); W_2(X); R_3(W); W_3(Y); R_4(W); R_4(Z); R_1(Z); W_1(Z)



T_1	T_2	T_3	T_4
$r_1(X)$			
	$r_2(X)$		
$w_1(Y)$			
		$r_3(Y)$	
	$r_2(Y)$		
	$w_2(X)$		
		$r_3(W)$	
		$w_3(Y)$	
			$r_4(W)$
			$r_4(Z)$
$r_1(Z)$			
$w_1(Z)$			

Repaso: Problemas de recuperabilidad

Recuperación

Se puede asumir que el Abort se implementa recuperando imágenes anteriores de los ítems.

Repaso: Problemas de recuperabilidad

Recuperación
Se puede asumir que el Abort se implementa recuperando imágenes anteriores de los ítems.

Ejemplos

- H_1 = w_1(X, 2); r_2(X); w_2(Y, 3); c_2.
  - Si T_1 aborta deberíamos abortar T_2 (violaríamos la semántica del *commit*)

w_1(X, 2) significa que la transaccion 1 escribe el valor 2 en X

Repaso: Problemas de recuperabilidad

Recuperación
Se puede asumir que el Abort se implementa recuperando imágenes anteriores de los ítems.

Ejemplos

- H_1 = w_1(X, 2); r_2(X); w_2(Y, 3); c_2.
  - Si T_1 aborta deberíamos abortar T_2 (violaríamos la semántica del *commit*)

- H_1 = w_1(X, 2); r_2(X); w_2(Y, 3); a_1.
  - Aborts en cascada

w_1(X, 2) significa que la transaccion 1 escribe el valor 2 en X

Repaso: Problemas de recuperabilidad

Recuperación
Se puede asumir que el Abort se implementa recuperando imágenes anteriores de los ítems.

Ejemplos

- H_1 = w_1(X, 2); r_2(X); w_2(Y, 3); c_2.
  - Si T_1 aborta deberíamos abortar T_2 (violaríamos la semántica del *commit*)
- H_1 = w_1(X, 2); r_2(X); w_2(Y, 3); a_1.
  - Aborts en cascada
- H_1 = w_1(X, 2); w_2(X, 3); a_1; a_2.
  - Problema para recuperar imagen

w_1(X, 2) significa que la transaccion 1 escribe el valor 2 en X

Niveles de recuperabilidad

Historia Recuperable RC

Una historia H es **RC** si siempre que una transacción T_i lee de T_j con $i \neq j$ en H y $c_i \in H$ entonces $c_j < c_i$.

Intuitivamente una historia es **recuperable** si una **transacción realiza commit solo después de que hicieron commit todas las transacciones de las cuales lee.**

Niveles de recuperabilidad

Historia Recuperable RC

Una historia H es **RC** si siempre que una transacción T_i lee de T_j con $i \neq j$ en H y $c_i \in H$ entonces $c_j < c_i$.

Intuitivamente una historia es **recuperable** si una **transacción realiza commit solo después de que hicieron commit todas las transacciones de las cuales lee.**

Avoids Cascading Aborts ACA

Una historia H es **ACA** si siempre que una transaccíon T_i lee X de T_j con $i \neq j$ en H entonces $c_j < r_i(X)$.
Lee solo valores de transacciones que ya hicieron *commit*.

Niveles de recuperabilidad

Historia Recuperable RC

Una historia H es **RC** si siempre que una transacción T_i lee de T_j con $i \neq j$ en H y $c_i \in H$ entonces $c_j < c_i$.

Intuitivamente una historia es **recuperable** si una **transacción realiza commit solo después de que hicieron commit todas las transacciones de las cuales lee.**

Avoids Cascading Aborts ACA

Una historia H es **ACA** si siempre que una transaccíon T_i lee X de T_j con $i \neq j$ en H entonces $c_j < r_i(X)$.
Lee solo valores de transacciones que ya hicieron *commit*.

Stricta ST

Una historia H es **ST** si siempre que $w_j(X) < o_i(X)$ con $i \neq j$ entonces $a_j < o_i(X)$ o $c_j < o_i(A)$ siendo $o_i(X)$ igual a $r_i(X)$ o a $w_i(X)$
No se puede leer ni escribir un ítem hasta que la transacción que lo escribió previamente haya hecho *commit* o *abort*.

Teorema de la recuperabilidad

$$ST \subset ACA \subset RC$$

Challenge Transacciones

Entren al siguiente link :)

https://docs.google.com/forms/d/e/1FAIpQLSeEu4c_ymRLn32pbgEiIrwQ8NdTY9AulQ8fQbp_96y18oI61Q/viewform?usp=header

Recuperación Ante Fallas

Estrategias de log

- Existen distintas estrategias posibles de gestión de la recuperación ante fallas:
  - **Método UNDO (actualización inmediata):** Se busca que los datos que una transacción modifica se **guarden** en disco lo antes posible, y obligatoriamente **antes del commit** de la transacción.
  - **Método REDO (actualización diferida):** Se busca que la transacción **no modifique nada en disco hasta que no haya hecho el commit**.
  - **Método UNDO/REDO:** El SGBD tiene la flexibilidad de guardar las modificaciones que hace la transacción en el momento que lo prefiera.

Estrategia UNDO (actualización inmediata)

---

1. Antes de modificar el registro en disco, vuelvo al log el valor <u>anterior</u>.

   (WRITE, T_i, X, v_{old})

1. El ítem modificado se vuelca a disco antes del commit de la transacción.
2. Cuando la transacción termina se escribe (COMMIT, T_i) en disco.

Recuperación bajo UNDO

---

1. Recorre el log desde el final hacia atrás, buscando transacciones que no tengan registrado su COMMIT o ABORT.
2. Para cada operación WRITE sin commit, se deshace la escritura colocando el valor v_{old}.
3. Se escribe (ABORT, T_i) por cada transacción sin commit. Luego, se vuelca el log a disco.

Estrategia REDO (actualización diferida)

---

1. El log indica <u>el nuevo valor</u> de ese ítem:

   (WRITE, T_i, X, v_{new})

1. Cuando la transacción termina se escribe(COMMIT, T_i) en disco.

2. En algún momento posterior, el SGBD decidirá volcar ese ítem a disco (¡es necesario asegurarse de que el Sist. Op. no lo haga antes del COMMIT!)

Recuperación bajo REDO

1. Recorre el log desde el final hacia atrás, buscando transacciones que **sí** tengan registrado su COMMIT.
2. Luego, desde el inicio del log hacia adelante, para cada operación WRITE de una transacción commiteada, se rehace la escritura, asegurando que el valor que había escrito la transacción se vuelque en disco.
3. Cuando se termina de recorrer el log hacia adelante, se escribe un registro de (ABORT, T_i) por cada transacción de la que no se encontró el commit, para darlas por cerradas. Para concluir, se vuelca el log a disco.

Puntos de control (checkpoints) en REDO

---

1. Se escribe un registro (BEGIN CKPT, t_{act}) en el log, registrando todas las transacciones que están activas en ese instante.
2. Se van volcando a disco todos los ítems modificados por transacciones commiteadas antes del BEGIN CKPT.
3. Al terminar, se escribe un registro de (END CKPT) en el log y se vuelca el log a disco.

Estrategia UNDO/REDO (actualización libre)

---

1. Se escribe un registro en el log indicando <u>tanto el nuevo valor del ítem como el viejo</u>:

   (WRITE, T_i, X, v_{old}, v_{new})

1. El registro de log del WRITE debe ser volcado a disco <u>antes</u> de que el ítem modificado se vuelque a disco. Esta es la regla maestra del UNDO/REDO.

2. El SGBD o el Sist. Op. podrán enviar el ítem a disco en cualquier momento, tanto antes como después del commit, siempre que se haya cumplimentado el paso 2.

Recuperación bajo UNDO/REDO

1. Recorre el log desde el final hacia atrás, dividiendo a las transacciones entre aquellas que tienen registrado su COMMIT y aquellas que no.
2. Para las que no tiene registrado su commit, a medida que encuentra escrituras, las va deshaciendo (volcando a disco el valor viejo que indica el log).
3. Luego, en una pasada desde el inicio del log hacia el final, reescribe en disco todos los ítems modificados por aquellas transacciones que registró como commiteadas.
4. Finalmente, se escribe un registro de (ABORT, T_i) por cada transacción de la que no se encontró el commit para darlas por cerradas, y se vuelca el log a disco.

Challenge Recuperación

---

https://forms.gle/estRSYJ32Nu3iEv88

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

1) Completar el paso 10.

2) Suponga un crash luego del paso 15. ¿Cuáles transacciones deberán rehacerse/deshacerse? ¿Por qué?

3) ¿Cuáles son los valores que quedan en los items X e Y luego del *recovery*? ¿Cómo debería quedar el log al final?

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	<T1, Y, 300, 3>
5	<T1,X, 200,8>
6	<START T3>
7	<ABORT T1>
8	<T3, X, 200, 15>
9	<COMMIT T2>
10	<START CKPT (**????**)>
11	<START T4>
12	<T4, Y , 300, 7>
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

1) Completar el paso 10.

Transacciones activas al momento del CKPT: T3

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	<T1, Y, 300, 3>
5	<T1,X, 200,8>
6	<START T3>
7	<ABORT T1>
8	<T3, X, 200, 15>
9	<COMMIT T2>
10	<START CKPT (**T3**)>
11	<START T4>
12	<T4, Y , 300, 7>
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

2) Suponga un crash luego del paso 15. ¿Cuáles transacciones deberán rehacerse/deshacerse? ¿Por qué?

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	<T1, Y, 300, 3>
5	<T1,X, 200,8>
6	<START T3>
7	<ABORT T1>
8	<T3, X, 200, 15>
9	<COMMIT T2>
10	<START CKPT (**T3**)>
11	<START T4>
12	<T4, Y , 300, 7>
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

2) Suponga un crash luego del paso 15. ¿Cuáles transacciones deberán rehacerse/deshacerse? ¿Por qué?

T1: ABORT T1 en paso 7, no se considera.

T2: completada antes del START CKPT, no se considera.

T3: falta COMMIT, deberá rehacerse.

T4: comienza luego del CKPT, deberá rehacerse.

No hay transacciones incompletas (a deshacer).

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	<T1, Y, 300, 3>
5	<T1,X, 200,8>
6	<START T3>
7	<ABORT T1>
8	<T3, X, 200, 15>
9	<COMMIT T2>
10	<START CKPT (**T3**)>
11	<START T4>
12	<T4, Y , 300, 7>
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

3) ¿Cuáles son los valores que quedan en los items X e Y luego del *recovery*? ¿Cómo debería quedar el log al final?

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	**<T1, Y, 300, 3>**
5	**<T1,X, 200,8>**
6	<START T3>
7	<ABORT T1>
8	**<T3, X, 200, 15>**
9	<COMMIT T2>
10	<START CKPT (**T3**)>
11	<START T4>
12	**<T4, Y , 300, 7>**
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Ejercicio - UNDO/REDO

Se dispone del siguiente **undo/redo** log con checkpoint no-quiescente.

3) ¿Cuáles son los valores que quedan en los items X e Y luego del *recovery*? ¿Cómo debería quedar el log al final?

X: 200 (original) -> 15 (por T3)

Y: 300 (original) -> 7 (por T4)

Log: rehacemos desde START CKPT

Paso	Registro
1	<START T2>
2	<T2, Z,100,20>
3	<START T1>
4	**<T1, Y, 300, 3>**
5	**<T1,X, 200,8>**
6	<START T3>
7	<ABORT T1>
8	**<T3, X, 200, 15>**
9	<COMMIT T2>
10	<START CKPT (**T3**)>
11	<START T4>
12	**<T4, Y , 300, 7>**
13	<COMMIT T3>
14	<COMMIT T4>
15	<END CKPT>

Repaso - DER

Parte 3: DER

Ejercicio de parcial

TD7: Ingeniería de Datos

Primer Parcial

Fecha: 7 de octubre de 2025	
Legajo:	
Apellido:	
Nombre:	

Bloque 1 Modelado	
Bloque 2 Normalización	
Bloque 2 Recuperabilidad	
Bloque 3 SQL 1	
Bloque 3 SQL 2	
Bloque 3 SQL 3	
Bloque 3 AR	
Bloque 4 Teórica 1	
Bloque 4 Teórica 2	
Bloque 4 Teórica 3	

EJERCICIOS A DESARROLLAR:

Bloque 1 Modelado

Diseñe el DER con sus restricciones adicionales. También el pasaje al modelo relacional.

a) Nos contratan para diseñar la base de datos de un proyecto muy ambicioso. El proyecto es realizar una página web que compita con YouTube a nivel Argentina y Latinoamérica. El nombre pensado por el equipo de marketing es **ArgenTube** y tendrá las siguientes características.

El sitio tendrá la posibilidad de registrar usuarios. El registro les pedirá su nombre y apellido, fecha de nacimiento y un password que se guardará encriptado. Cada usuario podrá subir videos. Éstos tendrán una categoría definida, cantidad de “me gusta” o “no me gusta” generados por los usuarios y número de reproducciones. Un usuario tiene la posibilidad de

Parte 3: DER

Diseñe el DER con sus restricciones adicionales. También el pasaje al modelo relacional.

a) Nos contratan para diseñar la base de datos de un proyecto muy ambicioso. El proyecto es realizar una página web que compita con YouTube a nivel Argentina y Latinoamérica. El nombre pensado por el equipo de marketing es ArgenTube y tendrá las siguientes características.

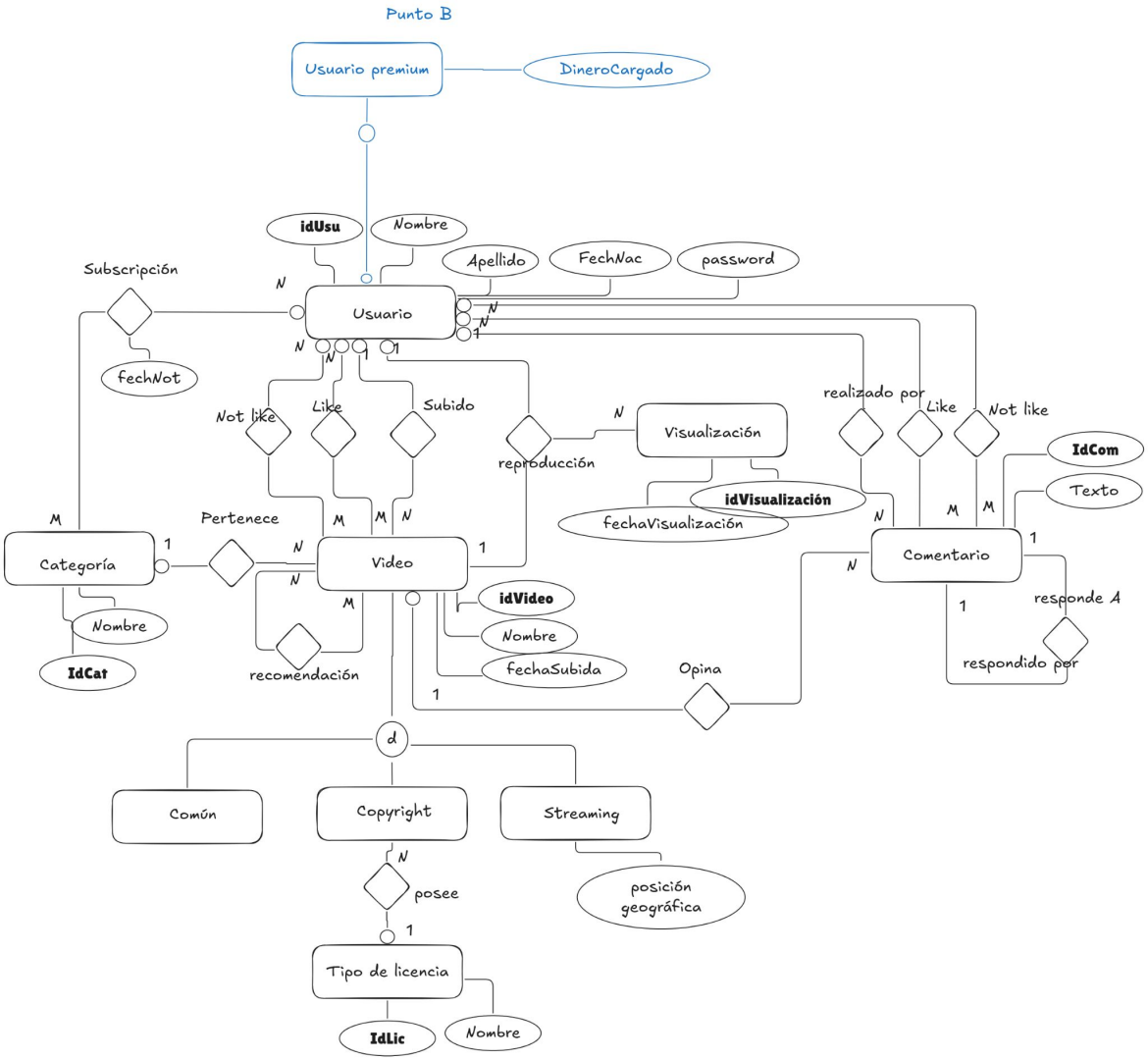
El sitio tendrá la posibilidad de registrar usuarios. El registro les pedirá su nombre y apellido, fecha de nacimiento y un password que se guardará encriptado. Cada usuario podrá subir videos. Éstos tendrán una categoría definida, cantidad de “me gusta” o “no me gusta” generados por los usuarios y número de reproducciones. Un usuario tiene la posibilidad de suscribirse a las distintas categorías. La idea es que el sistema le avise cuando se agrega un video nuevo de la categoría suscripta. Se debe poder guardar esa posibilidad en la base de datos.

También cada vídeo tendrá varios videos recomendados y varios comentarios generados por los usuarios. Cada comentario puede ser respondido por otro usuario y así sucesivamente. De cada comentario se debe guardar la fecha que se realizó y la posibilidad de que cada usuario pueda agregar “me gusta” o “no me gusta” al comentario.

Además de los videos comunes, éstos pueden tener copyright o ser generados vía streaming en vivo. Cuando un usuario quiere aplicar un tipo de licencia, al configurar el video elige entre una serie de licencias definidas. Cuando se generan vía streaming, se debe guardar la posición geográfica.

b) Después de la primera etapa, los inversionistas presionan para recuperar el dinero invertido. Observan que varios usuarios poseen mucha popularidad, como por ejemplo Christian de Lugano o La tigresa del Oriente, y proponen generar un nuevo tipo de usuario “Premium”. Éste tendrá la posibilidad de generar videos con streaming en vivo. Los usuarios comunes no podrán tener más esa posibilidad. Los nuevos usuarios Premium podrán cargar dinero que se les descontará de los minutos grabados en vivo. Dibuje y explique los cambios que posee el DER para soportar la nueva funcionalidad.

Parte 3: DER



Punto B)
Restricción adicional:
Sólo los usuarios premium pueden hacer videos de tipo “streaming”